HyMeKo: external hypergraph framework survey and matched measurement

Graph verification infrastructure

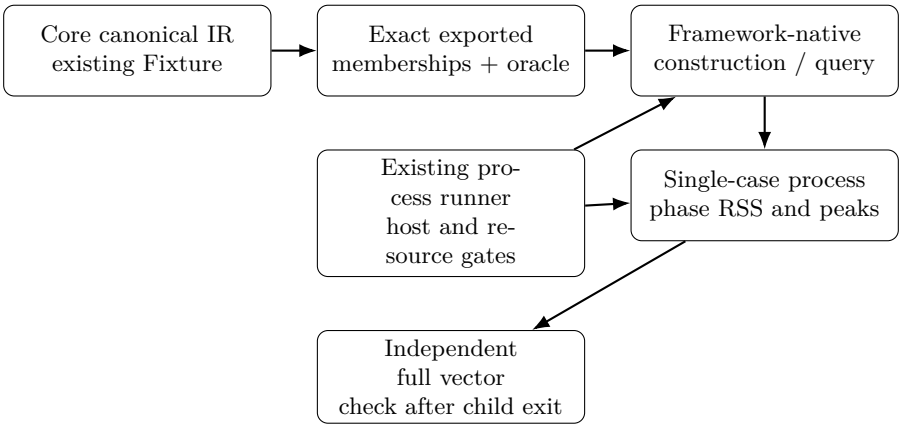
Created-at: 2026-10-21 19:49 Europe/Budapest

Scope. Evaluate HyperNetX 2.4.3, XGI 0.10.2, Hypergraphx 1.8.0 and Reticula 0.13.0 against the existing canonical-IR workloads. The user's request authorizes this performance survey. The user selected kato21 and enabled VPN; SSH with the pinned host key now works. Extend the comparison to graph searches: source reachability and unit-hyperedge-hop shortest distance from node 0 to the last declared node. Preserve the native HyMeKo Berge-path/Dijkstra controls.

ETA. One implementation/validation phase, followed by a bounded pilot and a fresh-process campaign. Package setup and adapter semantics determine the initial duration; expensive or incompatible cells stop at explicit limits. Do not promise a full campaign time before the pilot exposes per-call cost.

Owner. Graph owns external benchmark adapters and campaign orchestration. Core retains canonical IR construction and mathematical generators. Existing Fixture/IrConstruction exporters supply inputs; external adapters only construct each library's native object from those exact memberships. The manuscript owns presentation and receives results only after validation.

Discovery. Existing owners: `Cell`, `Evidence`, `ProcessRunner`, `Campaign`, `QuietHostGuard`, the shared `pagerank_support::Fixture`, and the thin canonical-IR gallery exporter. Extend these entry points; do not duplicate the IR builder, host gate or scheduler.



Semantic gates. Components compare complete normalized partitions over the declared node domain, including isolates. Detect dropped isolated nodes, duplicate hyperedge collapse and library caches before timing. Cold preparation and cached queries must have separate labels. PageRank must match the same vertex–hyperarc–vertex transition, multiplicity, self-return, teleportation, weights, damping and residual tolerance. A different centrality is not PageRank; an absent matching API is reported as not matched rather than silently replaced by our own algorithm under another library's name. Searches verify the complete reachable set and exact distance against canonical IR traversal. Label full-source distance APIs versus target queries, scalar distances versus returned paths, and explicit clique/linegraph projections.

Resources and timing. Fresh subprocesses, explicit opt-in, one algorithm thread initially, native/BLAS thread counts pinned. Preserve raw timings, package versions, source/binary/fixture hashes, exceptions and timeouts. Reuse the existing runner's 180-second process and 16-GiB memory bounds and host gate; individual operations have a 15-second admission bound. Start small and admit larger cells only after a successful bounded pilot. Target protocol: five fresh processes per admitted cell, two warm-ups, nine retained samples; report the actual batch/repetition scope. Do not compare Windows numbers directly to retained Linux medians. The seven prespecified families are chain, hub, eight regions, Steiner triples, complete 3-uniform, sunflower and Mycielski. Pilot every foreign component, reachability and shortest-distance call once; retain every successful pilot in the five-process prepared-query campaign. Preserve all admission failures.

Each fresh process also records its first query as a separate diagnostic of lazy projection cost; do not pool that single call with warmed samples.

Verification. Inspect installed primary-source APIs. Test isolated and disconnected nodes, repeated edges, empty inputs and small known partitions. Where ranking is supported, check the complete vector and an independent residual. Tests and import checks execute no timed workloads by default. Only actual current-host native runs qualify as timing evidence.

Change surface. Extend Graph's benchmark-only exporter and `scripts/performance_evidence/` adapters, runner selection, focused tests, isolated Python dependency lock and phase report. No production Rust algorithm, public API, Cargo dependency or CORE.YAML edit is planned. No commit, push, public release or third-party message is authorized by this task. After validation, update the owning manuscript in `D:/hakiko_ai_ws/01_analysis/zenodo/hymeko_01`: add a bounded external-query section, import verified CSV/provenance, extend its existing evidence reader and figure renderer, update the framework table and evidence totals, then rebuild the PDF and standalone Overleaf ZIP. Its origin was freshly fetched and HEAD is level with origin/main; existing local manuscript changes are preserved. Allow about 10–15 minutes after measurement for analysis, layout and packaging.

Risks and rollback. High-arity clique expansion can be quadratic and exceed memory; cached line-graphs can distort repeated-query timings; isolated nodes or duplicate edges may disappear during library construction. Fail closed or label an explicit adapter boundary. Keep the original artifacts and tests. The Python environment is isolated; removing task-owned adapter changes restores the old benchmark surface. User and other-agent modifications are preserved.

PageRank continuation: 23 October 2026, 21:27 Europe/Budapest

The user requests extending the manuscript's PageRank comparison and has enabled the kato21 VPN connection. Extend the existing adapters and guarded controller; reuse the existing canonical-IR exporter and retained native control binaries. No production Rust, dependency lock, or protected CORE.YAML item is changed.

Implementation and attribution. Add a benchmark-only ranking composition beside the existing routing composition. Reuse native construction through a shared composed-backend base. HyperNetX, XGI and Hypergraphx supply their public sparse incidence matrices; Reticula supplies public incidence lists. Preserve declared isolates and explicitly restore duplicate-hyperedge multiplicities in the composed operator when a native representation merges identical edges. Reject repeated members within a hyperedge in this set-incidence comparison.

For node-by-hyperarc incidence B, hyperarc multiplicities w, and cardinalities k, apply $Tx = B\,\mathrm{diag}(w/k)B^{\mathsf{T}}D^{-1}x$ through its factors, where $D = \mathrm{diag}(Bw)$. Uniform redistribution completes dangling columns. The existing SciPy dependency supplies LinearOperator and GMRES for $(I - 0.85T)x = 0.15p$. No pairwise product or new iterative solver is implemented. Use uniform initialization, restart at most 20, at most 200 inner iterations, and a conservative residual target derived from the common 10^{-10} L1 bound. Certify the final vector independently against the original exported memberships; record GMRES iterations and matrix-vector calls separately from native PageRank updates. All external rows are labeled framework + SciPy GMRES, not native PageRank.

Boundary and tests. Native hypergraph construction stays outside the timed query. Native incidence export, normalization, solver preparation, solving, full-vector result handling and temporary disposal are included. Retain the shared deadline, memory cap, host guard, explicit measurement opt-in and five-process protocol. Small tests compare against a direct dense linear solve and cover isolates, empty edges, duplicate edges, node permutations, incorrect vectors, failed solver convergence, record attribution and default-off execution. Validate the same tests on Linux before any pilot. The existing source-manifest checks must include every new adapter file. Frozen measured implementations remain separate from working source; use new report/output paths without modifying previous evidence.

Change surface and ETA. Existing plan; scripts/performance_evidence/ external.py, routing.py, campaign.py and their tests; new ranking.py and test_ranking.py; generated SYMBOLS.md if required. Local adapter and correctness work is one implementation phase, followed by a bounded pilot. Estimate the campaign duration from admitted cells rather than assuming every library fits the 15-second admission and 180-second process limits. Start with four graph families in original and permuted order; repeat native controls under the same host gate. Do not claim any added speed ratio until complete validation and fresh measurements exist. Failure leaves the current manuscript evidence unchanged.

Discovery. Existing external.py owns library construction and query protocol; routing.py owns composed route queries; campaign.py owns scheduling, limits and verification. The native ranking.rs remains the native Rust control. The new ranking.py owns only the external incidence-to-SciPy composition; its independent

small dense reference lives solely in test_ranking.py. No second IR, generator, scheduler, or native HyMeKo PageRank implementation is introduced.

Adapter correction, 23 October 2026, 22:40. Source review found that the HyperNetX and XGI query paths built a duplicate-edge histogram that they did not consume: both APIs preserve distinct hyperedge identities. Restrict this histogram to the simple-hypergraph containers that need it. Add a regression check preventing this unused work on native multiedge exports. All 59 local and Linux correctness checks pass. A repeated million-incidence pilot validates both changed paths. The 117 completed processes from the interrupted initial control remain diagnostic-only, with their original source archive; the publication campaign restarts with all 240 processes under one corrected frozen adapter. The IR inputs, native binaries, numerical solver and quiet-host gate are unchanged.

Isolated memory continuation: 24 October 2026

The user authorizes memory measurements on the same kato21 host. Extend the existing guarded campaign and benchmark clients; Graph owns this measurement infrastructure, while Core remains the sole IR construction owner. Reuse the approved frozen sources while the VPN prevents the Gitea fetch. No production algorithm, public API or dependency change is intended.

Split the verified canonical export into one immutable case per file. Each fresh process loads exactly one framework and one case; no native cross-framework parity graph or full-corpus JSON is resident. Release fixture/reference data after graph construction, retaining only metadata required to preserve duplicate hyperedges. Record imported-runtime baseline, input loading, graph construction, retained graph, algorithm preparation and five post-warm-up solves. Record resident RSS and reset Linux VmHWM at each phase using clear_refs=5; retain the lifetime maximum separately. These are process-residency observations, not allocator live-byte counts. Do not force allocator trimming or subtract independently measured peaks.

Use four graph families, original and seed-17 numbering, six workflows and five fresh processes per cell. Independent full-vector checks run in the controller after each child exits. Tests cover detached-input semantics, multiplicity, single-case input identity, malformed memory records and explicit default-off gates. Reuse CPU affinity, quiet-host checks, nice priority, time and memory limits. Build only the changed evidence client against retained production sources; verify their hashes. Estimated implementation and validation: 30–45 minutes; admitted measurement: about 10–20 minutes. Deliver raw evidence, a concise PDF with all frameworks and explicit memory boundaries, then close our remote processes.